

A Cut-Sensitive Attack on the First Open MIX Grammar Case

Exact parsing, infinite separation families,
and exhaustive evidence through length 21

Research report prepared for Vladimir Reshetnikov

September 20, 2026

Abstract

We investigate a concrete open problem in formal-language theory: does the three-component canonical tuple grammar introduced by Yuyama generate every word with equally many occurrences of $\mathbf{a}$, $\mathbf{b}$, $\mathbf{c}$? The problem is the first unresolved case of a combinatorial reformulation of the Kanazawa–Salvati conjecture. We develop a terminating, cut-sensitive membership recurrence and prove its equivalence to the defining grammar. We then prove a constructive three-component inclusion theorem and an infinite two-component obstruction family. In particular,

$$\mathbf{a}^{3m}\mathbf{b}^{3m}\mathbf{c}^m\mathbf{b}^{2m}\mathbf{c}^{4m}\mathbf{a}^{2m} \in L_3 \setminus L_2 \quad (m \geq 1).$$

The first member is a previously published example; the report supplies an analytic family argument rather than claiming that example as a discovery. A reproducible exhaustive computation accounts for all 417 019 279 balanced words of length at most 21, using symmetry reduction, and finds all of them in L_3 . Thus a counterexample, if one exists, has length at least 24, subject to the correctness of the supplied software execution. Python and C++ implementations, a forward-only validation oracle, and independently checked positive derivation certificates are included.

Research status. The chosen open problem is *not solved* here. The infinite-family results and parser characterization have complete proofs in this report; the finite frontier is a software-assisted result, not a proof-assistant-certified theorem. No bibliographic priority is claimed for the independently derived extensions.

Contents

1	The question, its scope, and the outcome	3
1.1	A small, explicit target within a larger conjecture	3
1.2	What this investigation establishes	3
1.3	Provenance and limits	4
2	The canonical tuple grammar	4
2.1	Regrouping means merging, not cutting	4
2.2	Elementary invariants and normalization	5
3	An exact backward membership recurrence	6
3.1	The three inverse operations	6
3.2	The characterization theorem	6
3.3	A diagnostic example: cuts cannot be forgotten	8
3.4	Complexity and implementation choices	8
4	A constructive three-component inclusion theorem	8
4.1	Balanced binary words and one-sided storage	8
4.2	Splitting the homogeneous block between both ends	9
4.3	Derivation certificates rather than search traces	10
5	An infinite family that cannot use two components	10
5.1	A balanced-factor obstruction	10
5.2	The obstruction theorem	11
6	Consequences: exact rank, counting, and rotation	13
7	Exhaustive computation and validation	14
7.1	Enumeration and symmetry weights	14
7.2	Completed finite results	14
7.3	A forward-only oracle checks the original rules	15
7.4	Structural and certificate tests	16
8	What the attempt says about the open problem	16
8.1	Why the natural obstruction strategy stops at level three	16
8.2	Valid and invalid reductions in a future search	16
8.3	A precise next mathematical target	17
9	Conclusion	17
A	Reference pseudocode	17
B	An explicit positive derivation for the first family member	18
C	Reproduction guide and artifact map	18
	References	20

1 The question, its scope, and the outcome

1.1 A small, explicit target within a larger conjecture

Let

$$\text{MIX} = \{w \in \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}^* : |w|_{\mathbf{a}} = |w|_{\mathbf{b}} = |w|_{\mathbf{c}}\}.$$

Here $|w|_x$ counts occurrences of the letter x . Membership in MIX is easy to decide by counting. The difficult question concerns *how a restricted grammar can construct all of its words*, not whether one can recognize balanced letter counts.

A multiple context-free grammar allows a nonterminal to produce a tuple of strings rather than a single string. Its arity, also called its fan-out, measures the number of components. Restrictions on how the components of several children may interleave lead to the well-nested subclass. Kanazawa and Salvati proved that MIX is not a tree-adjoining language [2]; Salvati proved that it is a two-component multiple context-free language without the well-nested restriction [3]. These statements concern different grammar classes and are not contradictory.

Yuyama’s September 12, 2026 preprint [1] gives explicit tuple sets $\mathcal{A}_r$, defined in section 2, and canonical languages $L_r = \text{concat}(\mathcal{A}_r)$. Its Section 6 and Remark 3.19 explicitly identify the following first unresolved case.

Problem 1.1 (The selected problem). Determine whether $L_3 = \text{MIX}$. Equivalently, either construct and prove a word in $\text{MIX} \setminus L_3$, or prove that every balanced word belongs to L_3 .

The source’s Corollary 3.18 identifies the full Kanazawa–Salvati conjecture with the assertion that $L_r \subsetneq \text{MIX}$ for *every* r . Consequently, a counterexample at $r = 3$ would establish one canonical case, not the full conjecture. Moreover, the cofinality theorem for arbitrary well-nested grammars can increase arity: proving $L_3 \neq \text{MIX}$ would not, by itself, exclude *every* well-nested grammar of fan-out three. Conversely, $L_3 = \text{MIX}$ would refute the full conjecture because the canonical grammar itself is well-nested. These logical directions are important.

1.2 What this investigation establishes

The principal mathematical result is a family with exact canonical rank three. Put $N = p + u$. For positive integers satisfying

$$p > s, \quad u > r, \quad r \neq s,$$

we prove

$$\mathbf{a}^p \mathbf{b}^{N-s} \mathbf{c}^r \mathbf{b}^s \mathbf{c}^{N-r} \mathbf{a}^u \in L_3 \setminus L_2. \tag{1.1}$$

The inequalities imply that every displayed block is nonempty. The proof rules out all two-component derivations by analyzing the only three possible initial useful cuts and all subsequent endpoint deletions. It is not an extrapolation from a finite search.

A stronger positive statement explains why this family cannot produce the sought three-component counterexample. Whenever $v \in \{\mathbf{b}, \mathbf{c}\}^*$ has N occurrences of each letter and $0 \leq p \leq N$, we construct the *specific tuple*

$$(\mathbf{a}^p, v, \mathbf{a}^{N-p}) \in \mathcal{A}_3.$$

The position of both cuts is part of the theorem.

The computational result is a completed exhaustive search through length 21, with no word outside L_3 . In particular, all 111 564 length-21 words rejected at level two are accepted at level

three. The enumeration uses symmetry and the inclusion $L_2 \subseteq L_3$; it does not run an independent level-three query on each of hundreds of millions of words.

1.3 Provenance and limits

The tuple grammar and the open status of theorem 1.1 are taken from [1], version 1. Its Example 1.4 already gives $\mathbf{aaabbbcbcccca} \in L_3 \setminus L_2$, and its Remark 3.19 reports that this is a lexicographically first shortest level-two obstruction. That example is a regression test, not a new discovery in this report. The one-sided binary inclusion used below also appears in the proof of its Proposition E.3; it is explicitly attributed when restated.

The proofs of the recurrence, the two-ended inclusion, the parameterized obstruction argument, and the associated consequences are given in full so that their correctness does not depend on trusting an experimental pattern. Their bibliographic novelty has not been established by an exhaustive literature review. The source and focused follow-up searches identify a published open target; they do not establish a claim that no unindexed solution exists anywhere.

2 The canonical tuple grammar

2.1 Regrouping means merging, not cutting

Write $\Sigma = \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}$, and let ε be the empty word. For a list $(v_1, \dots, v_m)$, define

$$\text{Regroup}_r(v_1, \dots, v_m) = \left\{ \left(\prod_{j=d_0+1}^{d_1} v_j, \dots, \prod_{j=d_{r-1}+1}^{d_r} v_j \right) : 0 = d_0 \leq d_1 \leq \dots \leq d_r = m \right\}. \quad (2.1)$$

An empty product denotes ε . Products here are ordered concatenations, not commutative products. An output component may merge consecutive input components; a component may not be split internally. For example, $(\mathbf{ab}, \mathbf{c})$ can be regrouped to $(\mathbf{abc}, \varepsilon)$, but not to $(\mathbf{a}, \mathbf{bc})$.

Definition 2.1 (Canonical grammar; Yuyama [1]). For an integer $r \geq 1$, $\mathcal{A}_r$ is the least set of r -tuples of words closed under the following rules.

- (1) **Empty:** $(\varepsilon, \dots, \varepsilon) \in \mathcal{A}_r$.
- (2) **Wrap:** from $(w_1, \dots, w_r) \in \mathcal{A}_r$, infer $(x_1 w_1 y_1, \dots, x_r w_r y_r)$, where each endpoint word x_i, y_i is either empty or a single letter, and their concatenation contains exactly one $\mathbf{a}$, one $\mathbf{b}$, and one $\mathbf{c}$.
- (3) **Insert and regroup:** from $U = (u_1, \dots, u_r)$ and $V = (v_1, \dots, v_r)$ in $\mathcal{A}_r$, choose $0 \leq i \leq r$ and infer any tuple in

$$\text{Regroup}_r(u_1, \dots, u_i, v_1, \dots, v_r, u_{i+1}, \dots, u_r).$$

Finally, $L_r = \{w_1 \cdots w_r : (w_1, \dots, w_r) \in \mathcal{A}_r\}$.

All components of the second child are inserted at a *single gap* in the first child's list. They are not arbitrarily shuffled through it. Likewise, the wrap rule has $2r$ distinct endpoint slots, each with capacity one. Treating a slot as capable of receiving a longer word changes the grammar.

2.2 Elementary invariants and normalization

For a tuple T , let $\text{nz}(T)$ be its list of nonempty components, with their order unchanged. A normalized tuple may have any number k of components with $0 \leq k \leq r$. Write $T \in \overline{\mathcal{A}}_r$ if padding it with empty components gives a tuple in $\mathcal{A}_r$.

Lemma 2.2 (Regrouping and empty positions). *The set $\mathcal{A}_r$ is closed under regrouping its own component list. Furthermore, all placements of empty components around a fixed ordered nonempty list are equivalent for membership. Thus $\overline{\mathcal{A}}_r$ is well defined, and*

$$w \in L_r \iff (w) \in \overline{\mathcal{A}}_r$$

for nonempty w ; the empty word corresponds to the empty normalized tuple.

Proof. Apply the third rule with the all-empty tuple as the second child. Inserting that child does not change the nonempty list, and the permitted regroupings include every merger of adjacent components and every desired placement of empty components. In particular, one can pass between two paddings of the same nonempty list without splitting anything. Merging all nonempty components proves the forward implication in the last equivalence; the reverse implication is the definition of concatenation. $\square$

Normalization does *not* forget the boundaries between nonempty components. It only forgets the positions of empty slots.

Proposition 2.3 (Balance, monotonicity, and symmetries). *Every tuple in $\mathcal{A}_r$ has a concatenation in MIX. Also $L_r \subseteq L_{r+1}$, $L_1 = \{\varepsilon\}$, and $\bigcup_{r \geq 1} L_r = \text{MIX}$. Membership is preserved by a global permutation of the alphabet and by full reversal*

$$(w_1, \dots, w_k) \mapsto (w_k^{\text{rev}}, \dots, w_1^{\text{rev}}).$$

Each L_r is closed under concatenation of words.

Proof. The empty tuple is balanced. A wrap adds one of each letter, and a binary rule adds the two children's letter counts. This proves balance by induction.

To increase arity, pad every tuple in a derivation with empty components. The same endpoint insertions remain legal, and the same nonempty component lists can be inserted and regrouped. This simulates the whole derivation at the larger arity. At arity one there are only two endpoint slots, so no wrap can insert three letters; binary combinations of empty tuples remain empty. Hence $L_1 = \{\varepsilon\}$.

For a balanced word of length $3N > 0$, use $3N$ components, one for each position of the word. Starting from empty components, fill one as-yet-unfilled occurrence of each letter in each of N wrap steps. The final tuple lists the individual letters of the word. This proves membership in L_{3N} , hence the union assertion.

A letter permutation maps each grammar rule to another legal rule. Full reversal reverses the ordered component list and each component, exchanges left and right endpoint insertions, and maps an insertion gap to its reflected gap. Regrouping is also respected. Induction proves both symmetry claims. Finally, normalize derivations of $x, y \in L_r$ to one nonempty component each, insert the second after the first, and merge the result. This derives $xy \in L_r$. $\square$

For a balanced word, define its *canonical rank* by $\rho(w) = \min\{r : w \in L_r\}$. This is a rank in this particular canonical family, not a minimal fan-out over all possible grammars generating a language. In particular, every singleton language is regular, so confusing these notions would make the definition meaningless.

3 An exact backward membership recurrence

3.1 The three inverse operations

Let $T = (t_1, \dots, t_k)$ be normalized, with $k \leq r$, and put $w = t_1 \cdots t_k$, $n = |w|$. We use half-open position intervals $[p, q)$, with $0 \leq p < q \leq n$.

Peeling. Choose one physical endpoint occurrence of each of **a**, **b**, **c**. Remove the three chosen letters and discard resulting empty components. A component can lose at most one letter from each end. A length-one component has one physical occurrence, not two different occurrences. Choosing one occurrence of each different letter automatically prevents choosing the same occurrence twice.

Extracting a balanced interval. Choose a proper, nonempty balanced factor $w[p : q]$. For each original component, intersect it with that interval. The ordered list of nonempty intersections is the inside tuple $B_{p,q}$. The outside tuple $A_{p,q}$ consists of the nonempty portions before and after that intersection, in their original order. Every original nonempty boundary is retained. If the interval lies strictly inside a single component, its exposed prefix and suffix become *two different components* of $A_{p,q}$. Reject this extraction alternative if either child has more than r nonempty components.

For example, extracting the displayed **abc** from the one-component tuple $(x \text{ abc } y)$, with nonempty words x, y , produces the outside tuple (x, y) , not (xy) . This distinction is necessary to reinsert the inside child at a gap.

Refining a cut. When $k < r$, split one component into two nonempty consecutive parts. This is a *backward* alternative: a successful derivation of the finer tuple can be merged to derive the current tuple. It is not a forward closure property of $\mathcal{A}_r$.

3.2 The characterization theorem

Theorem 3.1 (Cut-sensitive recurrence). *The following recursive procedure decides membership in $\overline{\mathcal{A}}_r$.*

1. *Accept the empty tuple. Reject tuples of more than r nonempty components, and reject tuples whose total letter counts are not balanced.*
2. *Accept if some peeling alternative has an accepted child.*
3. *Accept if some proper balanced interval extraction has both induced children accepted and each has at most r components.*
4. *Accept if some one-cut refinement has an accepted child.*
5. *Otherwise reject.*

The order of alternatives affects running time but not the answer.

Termination. Use the lexicographic measure

$$\mu(T) = (|t_1 \cdots t_k|, r - k) \quad \text{in } \mathbb{N} \times \mathbb{N}.$$

A peeling child has three fewer letters. Both extraction children have strictly fewer letters because the extracted factor is nonempty and proper. A refinement keeps the letter count unchanged but increases the number of components by one. Thus every recursive call strictly decreases μ . There are finitely many alternatives at each call, so the entire search terminates, even without memoization. $\square$

Soundness. Proceed by well-founded induction on μ . The empty tuple is legal. For a successful peel, pad the accepted child so that all surviving components occupy the positions they had before deletion. Components removed entirely can be restored as empty slots; there are at most r slots because the parent has at most r components. Inserting the three removed letters at their original endpoints is a valid wrap. A two-letter parent component whose letters were both deleted uses the two distinct endpoints of one empty child component, as required.

For a successful extraction, pad both accepted induced tuples to arity r . The inside components form a single consecutive block in the flattened parent. The outside tuple retains a boundary at the place where that block must be inserted, including the case of a split original component. Insert the inside list at that gap and regroup to the original tuple. Every original parent boundary is available in the combined list, so no internal splitting is needed.

For a successful refinement, the accepted finer tuple merges directly to the parent by theorem 2.2. These operations prove that every accepted tuple belongs to $\overline{\mathcal{A}}_r$. $\square$

Completeness. We show that every finite grammar derivation yields a successful recursive alternative. Empty placements may be ignored by theorem 2.2; regrouping is allowed as a derived unary rule.

Suppose the last effective operation is a wrap. Its three newly inserted letters occur at endpoints in the conclusion. Peeling precisely those occurrences recovers its premise, modulo empty positions. This is an alternative of the recurrence.

Suppose the last operation is a binary insertion with two nonempty children U, V , followed by regrouping. All letters from V form a contiguous factor $[p, q)$ in the concatenated conclusion. The factor is nonempty, proper, and balanced. Each component of V remains whole through the final regrouping. Consequently, the induced inside tuple $B_{p,q}$ is obtained by merging some adjacent components of V , possibly changing empty positions. Likewise, the induced outside tuple $A_{p,q}$ is a regrouping of U . If the conclusion has one component crossing both sides of the inserted factor, its outside prefix and suffix come from opposite sides of an actual gap in U ; retaining them as separate components therefore does not create more components than U had. Both induced children belong to $\overline{\mathcal{A}}_r$ by regrouping closure, have at most r components, and have smaller length. Induction applies to them.

A binary insertion with an empty child is just regrouping of its nonempty premise. If the normalized conclusion is unchanged, discard that redundant step. Otherwise its premise is a strict refinement of its conclusion, obtainable by inserting one missing boundary at a time. All intermediate tuples are coarsenings of the derivable premise, so all are derivable. Every inserted boundary is a legal one-cut refinement, and the measure decreases along this inverse sequence. There cannot be an infinite chain of discarded redundant rules because the original derivation is finite. These cases exhaust the defining rules, proving completeness. $\square$

3.3 A diagnostic example: cuts cannot be forgotten

Example 3.2. The word `aaccbb` belongs to L_2 , but

$$(a, accb, b) \notin \mathcal{A}_3.$$

For the positive assertion, `ccbb` is a balanced binary word, so the one-sided inclusion of [theorem 4.1](#) applies. For the negative assertion, no component endpoint is `c`, and there is no spare component for refinement. No length-three factor of `aaccbb` is balanced; the only other nonempty balanced factor length is six, which is the whole word. Therefore every recurrence alternative fails.

In particular, replacing tuple states by their concatenations gives a wrong parser. Even accepting every cut of an accepted word is already wrong at total length six. This small example is included in the tests.

3.4 Complexity and implementation choices

For a fixed input word of length n , every component encountered by the uncanonicalized recurrence is an interval of the original word. Deleting endpoints shortens an interval, while extraction and refinement split intervals without reordering their surviving positions. With k nonempty components, the number of possible ordered disjoint interval lists is

$$\binom{n+k}{2k}.$$

Indeed, the k interval lengths are positive and the $k+1$ gaps are nonnegative; subtracting one from each interval length gives this stars-and-bars count. Hence the total number of positional states is $O_r(n^{2r})$.

Each state has $O_r(1)$ endpoint choices, $O(n^2)$ interval choices, and $O_r(n)$ refinement choices. A straightforward implementation that materializes child strings in $O_r(n)$ time therefore has the conservative bounds

$$O_r(n^{2r+3}) \text{ time,} \quad O_r(n^{2r+1}) \text{ stored string space}$$

with an unbounded memo table and ordinary constant-time dictionary operations apart from string processing. A dense positional chart can avoid relying on hashing for the state lookup bound. These bounds are not claims of optimal grammar parsing complexity.

The supplied programs additionally identify states under global alphabet renaming and full reversal. These are genuine symmetries by [theorem 2.3](#); at most twelve positional variants are involved. Python uses an optional least-recently-used cache. C++ caches states of letter length at most 18, and the enumeration driver occasionally clears its level-two cache. Cache eviction may cause recomputation; it never licenses accepting or rejecting a state without evaluating it. The production exhaustive driver has no node budget or timeout that could be mistaken for a negative membership result.

4 A constructive three-component inclusion theorem

4.1 Balanced binary words and one-sided storage

A binary word $v \in \{\mathbf{b}, \mathbf{c}\}^*$ is called balanced here when $|v|_{\mathbf{b}} = |v|_{\mathbf{c}}$. Let this common number be N . For a nonempty balanced binary word, follow the difference $\#\mathbf{b} - \#\mathbf{c}$ along its prefixes. Its first

return to zero gives either a proper balanced initial factor or the whole word. In the latter case the first and last letters differ, and deleting them leaves a balanced word. Thus induction can always use either a concatenation of two shorter balanced words or an outer pair of opposite letters.

Lemma 4.1 (One-sided binary inclusion). *For every balanced binary word v with N occurrences of each letter,*

$$(v, \mathbf{a}^N) \in \mathcal{A}_2, \quad (\mathbf{a}^N, v) \in \mathcal{A}_2.$$

This fact is also established in the proof of Yuyama's Proposition E.3 [1]; we give the elementary construction for use below.

Proof. Induct on N . The empty case is immediate. If the first return is the whole word, write $v = xty$, where $\{x, y\} = \{\mathbf{b}, \mathbf{c}\}$ and t is balanced with $N - 1$ occurrences of each letter. From $(t, \mathbf{a}^{N-1})$, insert x and y at the two ends of the first component and $\mathbf{a}$ at one end of the second. This is one wrap.

Otherwise write $v = v_1v_2$ with positive sizes N_1, N_2 , $N_1 + N_2 = N$. Induction gives $(v_1, \mathbf{a}^{N_1})$ and $(v_2, \mathbf{a}^{N_2})$. Insert the second tuple between the two components of the first and regroup:

$$(v_1, v_2, \mathbf{a}^{N_2}, \mathbf{a}^{N_1}) \mapsto (v_1v_2, \mathbf{a}^{N_2+N_1}).$$

This proves the first assertion. Apply full reversal to the first assertion for v^{rev} to obtain the second. $\square$

4.2 Splitting the homogeneous block between both ends

Theorem 4.2 (Two-ended binary inclusion). *Let $v \in \{\mathbf{b}, \mathbf{c}\}^*$ contain N copies of each letter. For all $p, u \geq 0$ with $p + u = N$,*

$$(\mathbf{a}^p, v, \mathbf{a}^u) \in \mathcal{A}_3. \tag{4.1}$$

The proof yields a finite derivation using only the original three rules.

Proof. Induct on N , including all values of p at once. At $N = 0$, all components are empty. For $N > 0$, use the first-return decomposition of v .

If the first return is the whole word, write $v = xty$, with opposite binary letters x, y . If $p > 0$, apply the induction hypothesis to $(\mathbf{a}^{p-1}, t, \mathbf{a}^u)$, then insert $\mathbf{a}$ in the first component and x, y at the ends of the middle component. If $p = 0$, then $u = N > 0$; start from $(\varepsilon, t, \mathbf{a}^{u-1})$ and put the new $\mathbf{a}$ in the last component instead. The insertions use three distinct legal slots.

Otherwise write $v = v_1v_2$, with balanced positive sizes N_1, N_2 . There are two cases; choosing the allocation carefully is what avoids a forbidden crossing of child components.

If $p \leq N_1$, use the inductively derivable tuples

$$X = (\mathbf{a}^p, v_1, \mathbf{a}^{N_1-p}), \quad Y = (\varepsilon, v_2, \mathbf{a}^{N_2}).$$

Insert Y after the second component of X . The resulting list is

$$(\mathbf{a}^p, v_1, \varepsilon, v_2, \mathbf{a}^{N_2}, \mathbf{a}^{N_1-p}).$$

Group its first entry, its next three entries, and its last two entries. The result is $(\mathbf{a}^p, v_1v_2, \mathbf{a}^{N-p})$.

If $p > N_1$, take instead

$$X = (\mathbf{a}^{p-N_1}, v_2, \mathbf{a}^{N-p}), \quad Y = (\mathbf{a}^{N_1}, v_1, \varepsilon).$$

Insert Y after the first component of X . The combined list is

$$(\mathbf{a}^{p-N_1}, \mathbf{a}^{N_1}, v_1, \varepsilon, v_2, \mathbf{a}^{N-p}).$$

Group its first two entries, its next three entries, and its last entry. This again gives the required tuple. All exponents used in the induction hypotheses are nonnegative, and both binary words are shorter. This completes the construction. $\square$

Corollary 4.3. *Every balanced word in the pattern $\mathbf{a}^*\{\mathbf{b}, \mathbf{c}\}^*\mathbf{a}^*$ belongs to L_3 . The same is true after any global letter permutation.*

This corollary rules out a large natural source of prospective three-component counterexamples: arbitrarily complicated binary interiors with all occurrences of the third letter confined to the two ends. Complexity of the binary interior alone cannot defeat level three.

4.3 Derivation certificates rather than search traces

The construction is implemented in `src/family_certificate.py`. Its output records actual empty, wrap, and binary-regroup rules, including insertion gaps and weak cut indices. The independent checker `src/check_certificate.py` simply verifies these local rules; it does not invoke the membership parser or the family theorem.

There is one wrap for each pair of binary letters. The recursive first-return decomposition uses at most $N - 1$ binary nodes and at most N empty leaves when $N > 0$. Thus a tree certificate can be chosen with at most $3N - 1$ nodes. The number of rule nodes is linear, although explicitly storing the full tuple strings at every node can take quadratic total space. This distinction is useful when turning the construction into a proof-producing implementation.

5 An infinite family that cannot use two components

5.1 A balanced-factor obstruction

Call a nonempty word *factor-balanced-primitive* if it is balanced and has no nonempty proper balanced factor. This terminology refers to letter balance, not to the usual notion of a word that is not a proper power.

Lemma 5.1 (A homogeneous exterior and a positive binary excursion). *Suppose $z \in \{\mathbf{b}, \mathbf{c}\}^*$ has $M > 0$ copies of each letter, and every nonempty proper prefix of z has more $\mathbf{b}$'s than $\mathbf{c}$'s. If $P, U \geq 0$ and $P + U = M$, then $\mathbf{a}^P z \mathbf{a}^U$ is factor-balanced-primitive.*

Proof. A nonempty balanced factor must contain an $\mathbf{a}$, since a word without $\mathbf{a}$'s cannot have three equal positive letter counts. If the factor meets both exterior $\mathbf{a}$ -blocks, it contains all of z . It must therefore contain M occurrences of $\mathbf{a}$, hence all occurrences of every letter, and so is the whole word.

If it meets only the initial $\mathbf{a}$ -block, its binary part is a nonempty prefix of z . That prefix must have equal binary counts. By the strict prefix assumption it is all of z , so the factor needs M initial $\mathbf{a}$'s. This is possible only when $U = 0$, and then the factor is the whole word. The suffix case is similar: a proper nonempty suffix of z cannot have equal binary counts, because its complementary proper prefix would then have zero balance. These cases exhaust every factor containing $\mathbf{a}$. $\square$

A useful special case is

$$z = \mathbf{b}^Q \mathbf{c}^R \mathbf{b}^S \mathbf{c}^T, \quad Q + S = R + T = M, \quad Q > R, \quad T > S, \quad (5.1)$$

where $Q, T > 0$ and $R, S \geq 0$. Its binary height is positive between its two endpoints: after the first descending block it is $Q - R > 0$, after the next ascending block it is T , and only the last descending block returns it to zero. Either R or S may be zero; the argument still holds.

5.2 The obstruction theorem

Theorem 5.2 (A parameterized level-two obstruction). *Let p, u, r, s be positive integers, let $N = p + u$, and assume*

$$p > s, \quad u > r, \quad r \neq s.$$

Then the balanced word

$$W_{p,u,r,s} = \mathbf{a}^p \mathbf{b}^{N-s} \mathbf{c}^r \mathbf{b}^s \mathbf{c}^{N-r} \mathbf{a}^u \quad (5.2)$$

satisfies $W_{p,u,r,s} \notin L_2$.

Proof. Set $q = N - s$ and $t = N - r$. The hypotheses imply

$$N > r + s, \quad q > r, \quad t > s.$$

Reversing the entire word and interchanging $\mathbf{b}$ with $\mathbf{c}$ sends (p, u, r, s) to (u, p, s, r) . This preserves all hypotheses and membership. Since $r \neq s$, we may therefore assume

$$r < s. \quad (5.3)$$

In particular $p > s > r$, $u > r$, and both exterior blocks survive any sequence of at most r deletions of $\mathbf{a}$.

The binary interior $\mathbf{b}^q \mathbf{c}^r \mathbf{b}^s \mathbf{c}^t$ has the form (5.1). By [theorem 5.1](#), the whole word has no nonempty proper balanced factor. Viewed as a one-component tuple, its endpoints are both $\mathbf{a}$, so it cannot be peeled. The recurrence of [theorem 3.1](#) must therefore begin by inserting one cut.

After that cut there are two nonempty components and no spare component slot. To peel a triple, the cut must expose both a $\mathbf{b}$ and a $\mathbf{c}$, since the two outer endpoints are $\mathbf{a}$'s. The only useful initial cuts are the following three seams:

$$\begin{aligned} \text{I: } & (\mathbf{a}^p \mathbf{b}^q, \mathbf{c}^r \mathbf{b}^s \mathbf{c}^t \mathbf{a}^u), \\ \text{II: } & (\mathbf{a}^p \mathbf{b}^q \mathbf{c}^r, \mathbf{b}^s \mathbf{c}^t \mathbf{a}^u), \\ \text{III: } & (\mathbf{a}^p \mathbf{b}^q \mathbf{c}^r \mathbf{b}^s, \mathbf{c}^t \mathbf{a}^u). \end{aligned}$$

Every other cut has an endpoint alphabet missing at least one letter. It cannot be extracted, peeled, or further refined, so it is immediately rejected. We now rule out every continuation of the three useful cuts.

Case I. Until $j = r$ triples have been peeled, the **b** must be removed from the end of the first component, and the **c** from the start of the second. After j steps, the flattened binary interior is

$$\mathbf{b}^{q-j} \mathbf{c}^{r-j} \mathbf{b}^s \mathbf{c}^t.$$

The two relevant strict inequalities are

$$(q - j) - (r - j) = q - r > 0, \quad t - s > 0.$$

It remains a positive binary excursion; the exterior **a**-counts sum to its binary size. Thus [theorem 5.1](#) excludes every extraction throughout the trajectory. Both exterior **a**-blocks remain positive because $p, u > r$, and neither component becomes empty. At $j = r$, the first **c**-block disappears. The two inner endpoints are then both **b**; the outer endpoints remain **a**. No endpoint is **c**, so peeling stops without reaching the empty tuple.

Case II. The forced binary deletions now remove from the first **c**-block and the second **b**-block. After $0 \leq j \leq r$ steps the binary interior is

$$\mathbf{b}^q \mathbf{c}^{r-j} \mathbf{b}^{s-j} \mathbf{c}^t.$$

Its strict inequalities are

$$q - (r - j) = q - r + j > 0, \quad t - (s - j) = t - s + j > 0.$$

Again every flattened word is factor-balanced-primitive, both exterior blocks stay positive, and neither component disappears. Because $r < s$, the first **c**-block disappears first, at $j = r$. Both inner endpoints are then **b**, while both outer endpoints are **a**. Peeling again stops with no **c**-endpoint.

Case III. For $0 \leq j \leq s$, the binary deletions remove from the second **b**-block and the last **c**-block. The binary interior is

$$\mathbf{b}^q \mathbf{c}^r \mathbf{b}^{s-j} \mathbf{c}^{t-j},$$

with

$$q - r > 0, \quad (t - j) - (s - j) = t - s > 0.$$

The initial **a**-block stays nonempty because $p > s$. The terminal **a**-block might vanish before these s steps have finished. This causes no omitted alternative: it merely exposes the other end of the *same* last **c**-block. Removing a **c** from either exposed end gives the same shorter homogeneous block. The **b**-deletion is still forced from the second **b**-block, and all available **a**-deletions are already covered by allowing arbitrary allocations between the two exterior blocks.

The positive-excursion inequalities still exclude every extraction. At $j = s$, the second **b**-block disappears while $t - s > 0$. Both components are nonempty: the first still contains **a**, $\mathbf{b}^q$, and $\mathbf{c}^r$, and the second still contains $\mathbf{c}^{t-s}$. Their exposed endpoints are **a** or **c**; no endpoint is **b**. No further peel is possible.

In all three cases, every state before the terminal stall has two nonempty components and no proper balanced factor. There is no way to free a component slot or to use a binary extraction. The only possible peeling trajectories all stall. The complete recurrence therefore rejects every initial cut, and hence rejects the word itself. $\square$

Remark 5.3 (Why the unequal parameters matter). The assumption $r \neq s$ is not cosmetic. If $r = s$, the middle two blocks can disappear simultaneously in Case II, exposing different endpoints and invalidating the stall argument. The theorem asserts a sufficient obstruction region; it does not assert that its conditions are a complete classification of all six-block words.

6 Consequences: exact rank, counting, and rotation

Corollary 6.1 (Exact canonical rank three). *Every word in theorem 5.2 has $\rho(W_{p,u,r,s}) = 3$. In particular,*

$$w_m = \mathbf{a}^{3m} \mathbf{b}^{3m} \mathbf{c}^m \mathbf{b}^{2m} \mathbf{c}^{4m} \mathbf{a}^{2m} \quad \text{satisfies} \quad \rho(w_m) = 3 \quad (m \geq 1). \quad (6.1)$$

Proof. The binary interior of $W_{p,u,r,s}$ has N copies of each binary letter. The two-ended inclusion theorem gives membership in L_3 , while theorem 5.2 excludes L_2 . Monotonicity excludes every lower rank. For w_m , take $(p, u, r, s) = (3m, 2m, m, 2m)$. $\square$

The word w_1 is the published example described in section 1. The proof above treats all m , and in fact a larger multi-parameter region, without relying on the published computer search for its negative half.

Corollary 6.2 (A cubic-size certified obstruction region). *For $N \geq 2$, the parameter region in theorem 5.2 gives exactly*

$$C_N = \binom{N-1}{3} - \left\lfloor \frac{(N-2)^2}{4} \right\rfloor \quad (6.2)$$

distinct displayed six-block words of length $3N$, with the binomial coefficient interpreted as zero when its upper index is below three. Thus $|(L_3 \setminus L_2) \cap \Sigma^{3N}| \geq C_N$, and $C_N \sim N^3/6$.

Proof. For fixed p and $u = N - p$, choose $1 \leq s \leq p - 1$ and $1 \leq r \leq u - 1$, excluding $r = s$. The number is $(p - 1)(u - 1) - \min(p - 1, u - 1)$. Put $M = N - 2$ and $x = p - 1$. Summing over $x = 1, \dots, M - 1$ gives

$$\sum_{x=1}^{M-1} x(M-x) = \binom{M+1}{3}, \quad \sum_{x=1}^{M-1} \min(x, M-x) = \lfloor M^2/4 \rfloor.$$

The first identity follows from the sums of the first integers and their squares; the second follows by pairing positions equidistant from the ends, with a central term when M is even. All six runs are nonempty and adjacent runs have different letters, so their run lengths uniquely recover the parameters. There is no duplication in this count. The asymptotic follows from (6.2). $\square$

For $N = 5, 6, 7, 8, 9, 10$, the lower bounds are 2, 6, 14, 26, 44, 68. These are counts in one oriented block pattern, not total counts of level-two nonmembers; the exhaustive totals in section 7 are much larger.

Corollary 6.3 (Failure of cyclic-rotation closure). *The canonical language L_2 is not closed under cyclic rotation.*

Proof. Write $w_m = \mathbf{a}^{3m} v_m \mathbf{a}^{2m}$. The word $\mathbf{a}^{5m} v_m$ belongs to L_2 by theorem 4.1, because v_m has $5m$ copies of each binary letter. Moving the first $2m$ occurrences of $\mathbf{a}$ from the beginning to the end produces w_m , which is not in L_2 by theorem 6.1. $\square$

This is a property of one canonical language. It is not a claim that the entire class of well-nested multiple context-free languages lacks any particular closure property.

Proposition 6.4 (What uniform scaling does and does not imply). *Let h_m replace each letter x by x^m , with $m \geq 1$. Componentwise application of h_m preserves $\mathcal{A}_r$, and therefore $h_m(L_r) \subseteq L_r$.*

Proof. Induct on a derivation. Empty and binary-regroup rules commute with the homomorphism. Replace each wrap by m copies of that same wrap, using the same endpoint slots. Repeatedly inserting x before a component and y after it yields $x^m h_m(w) y^m$, exactly the image of that component's original wrap. Each repeated step is legal. $\square$

This only propagates *positive* membership. It does not establish $h_m(w) \notin L_r$ from $w \notin L_r$. The negative scaling statement for (6.1) genuinely uses [theorem 5.2](#); it is not an automatic closure argument.

7 Exhaustive computation and validation

7.1 Enumeration and symmetry weights

The number of words with N occurrences of each of three letters is

$$M_N = \frac{(3N)!}{(N!)^3}. \quad (7.1)$$

For $N > 0$, the action of the six alphabet permutations is free: a permutation fixing a word must fix every letter appearing in it, and all three appear. Choose the representative in which the first new letter is **a**, the second is **b**, and the third is **c**. The Python enumerator uses exactly these representatives, each with weight six.

The C++ enumerator also uses full reversal. Let $R(w)$ be the reverse word relabelled by first occurrence. It visits only representatives satisfying $w \leq R(w)$ lexicographically. The corresponding orbit has size six if $w = R(w)$, and size twelve otherwise. This is because reversal either stays in the original six-element letter orbit or creates a disjoint second letter orbit. The empty word is handled separately with weight one.

For every length, the program checks that the sum of these weights is exactly (7.1). At each representative it first queries L_2 . Only a negative level-two result triggers a level-three query, since $L_2 \subseteq L_3$ is proved independently of the program. Counts of negative results are multiplied by the full orbit weight.

7.2 Completed finite results

Length	All balanced words	C++ symmetry orbits	Outside L_2	Outside L_3
0	1	1	0	0
3	6	1	0	0
6	90	11	0	0
9	1 680	148	0	0
12	34 650	2 955	0	0
15	756 756	63 231	84	0
18	17 153 136	1 430 912	3 672	0
21	399 072 960	33 259 920	111 564	0

Table 1: Exact finite enumeration using the proved recurrence. Counts are software-assisted results. The empty word is included.

The totals sum to 417 019 279 balanced words. The final C++ run was completed on September 20, 2026. Its machine-readable output is `data/cpp_final_through21.json`; the corresponding progress and completion log is `data/cpp_final_exhaust.log`. A separately implemented Python enumerator and parser reproduce all the counts through length 18, without using reversal to reduce the enumeration. Its output is `data/python_through18.json`.

The first search at length 21 and the final readability-refactored C++ version also agree. Timings are stored in the logs rather than presented as portable performance guarantees. The final programs use ordinary exact integer counts and Boolean membership states; there are no floating-point decisions about membership.

Proposition 7.1 (Software-assisted finite frontier). *The supplied exhaustive runs return*

$$\text{MIX} \cap \Sigma^{\leq 21} \subseteq L_3.$$

Subject to the correctness of the implementation and its recorded execution, every word in $\text{MIX} \setminus L_3$ has length at least 24.

Derivation from the recorded finite computation. The enumeration accounts for every balanced word at the possible lengths 0, 3, 6, $\dots$, 21, by the orbit-weight argument and the checked multinomial totals. No representative is rejected at level three, with level-two positives transferred by monotonicity. By theorem 3.1, the implemented recurrence is the exact mathematical decision procedure. Every balanced length is a multiple of three, so the next possible length is 24. The remaining trust assumption is the faithful implementation and execution of that procedure, not a probabilistic inference from a sample. $\square$

7.3 A forward-only oracle checks the original rules

Two ports of the same incorrect recurrence could agree. We therefore also implemented a deliberately different small-instance oracle: forward saturation of the three rules of theorem 2.1, keeping all exact r -tuples, including empty components. At total length n , a wrap comes from length $n - 3$, and a binary rule with two nonempty children comes from smaller positive lengths. Empty-child binary rules are accounted for by a final regrouping closure, which is idempotent. This computes every forward-derivable tuple through length six without using inverse interval extraction.

Arity	Length	Exact tuples tested	Forward accepted	Disagreements
1	6	90	0	0
2	3	24	24	0
2	6	630	594	0
3	3	60	60	0
3	6	2 520	2 502	0

Table 2: Selected forward-oracle checks. The JSON additionally records the empty cases and arity-one length-three case.

For each balanced word of a tested length, every weak cut sequence into r components is compared against the backward parser. The 18 rejected exact three-tuples of length six are a useful check that the code has not silently erased component boundaries. The example in theorem 3.2 belongs to this set.

7.4 Structural and certificate tests

The structural tests verify the published length-15 witness, the cut-sensitive length-six example, invalid-input handling, and every member of the obstruction parameter region for $N \leq 12$. They also construct and locally check 1 965 positive certificates: every balanced binary word with $N \leq 5$, for every allocation $p = 0, \dots, N$. A deliberately corrupted wrap is rejected by the certificate checker.

These checks serve different purposes. Forward saturation tests the recurrence against the original grammar on small cases. The Python and C++ agreement tests two implementations and two symmetry enumerators over a much larger word domain. The local certificate checker validates particular positive derivations without searching. None of these is a machine-checked proof of the universal analytic theorems. Nor are stored positive certificates supplied for every orbit in the length-21 run; the finite frontier is independently reproducible computation, not a compact formal certificate of all its decisions.

8 What the attempt says about the open problem

8.1 Why the natural obstruction strategy stops at level three

The proof of theorem 5.2 works because a two-component derivation, once cut, has no unused component slot. A positive binary excursion prevents nontrivial extraction, and endpoint deletion eventually exposes only two letters. The argument would fail if another useful cut could be inserted before that stall. At level three, that extra cut is available, and theorem 4.2 does more than suggest a rescue: it explicitly derives the whole two-ended binary family.

Consequently, simply making the published obstruction longer, using unequal run lengths, or substituting a more complicated balanced binary interior cannot solve theorem 1.1 while retaining the pattern $a^*\{b, c\}^*a^*$. A successful obstruction must leave this entire positive family, up to alphabet permutation.

The endpoint argument is nevertheless useful. It demonstrates how to prove a *negative* grammar-membership statement without relying on the parser: identify all initial meaningful cuts, establish an invariant excluding binary extraction, and show that every legal endpoint trajectory stalls while keeping all component slots occupied. A level-three version would need to control two initial cuts and also all ways that one of three components can vanish. The present proof does not supply that stronger invariant.

8.2 Valid and invalid reductions in a future search

The concatenation closure in theorem 2.3 gives a sound reduction: a shortest word outside L_r , if there is one, has no nonempty proper balanced prefix. Otherwise its balanced prefix and suffix are shorter and hence both in L_r , forcing their concatenation into L_r .

The superficially stronger reduction to words having *no proper balanced factor anywhere* has not been justified. An internal factor can be extracted only when its two induced tuple children are accepted; acceptance of the flattened outside word is insufficient. The cut trap in theorem 3.2 shows exactly why. Thus searching only factor-balanced-primitive candidates is a heuristic restriction, not an exhaustive reduction of the open problem.

The same warning applies to cyclic rotation: theorem 6.3 shows that even level two does not respect it. Alphabet renaming and full reversal are safe symmetries; arbitrary cyclic rotation is not

used in the exhaustive enumeration.

8.3 A precise next mathematical target

One route would be to find a family of three-component tuple states with a finite symbolic description, closed under every possible backward peeling and admissible extraction, whose states cannot reach the empty tuple. The family must also account for all refinements of a one-component candidate. This is stronger than finding a few difficult strings or observing a long running time.

A complementary route toward equality would be an induction asserting membership of a suitably rich class of *tuples*, not merely all balanced words. The theorem $(\mathbf{a}^p, v, \mathbf{a}^u) \in \mathcal{A}_3$ is an example of an induction hypothesis strong enough to survive concatenation of binary interiors. Extending it to arbitrary distributions of all three letters would require additional structure; arbitrary cut placements cannot be included because they are already false at length six.

The finite frontier offers no statistical reason to prefer equality or inequality. Many grammar obstructions require a structured word far longer than every short test case. What it supplies is a checked starting point and a lower bound on the length of a prospective witness, not a resolution by numerical evidence.

9 Conclusion

The selected open problem has led to three kinds of concrete output. First, the grammar admits an exact terminating backward recurrence whose state records the actual component boundaries. Second, the report gives complete analytic proofs of a broad three-component positive family and a multi-parameter two-component negative family, with exact-rank, counting, and rotation consequences. Third, two implementations and several independent validation mechanisms support an exhaustive frontier through length 21.

The result is a partial research contribution and a reproducible experimental platform. It is *not* a solution of $L_3 \stackrel{?}{=} \text{MIX}$, not a resolution of the full Kanazawa–Salvati conjecture, and not a formal verification in a proof assistant. The positive family identifies a substantial region where a counterexample cannot lie; the obstruction proof identifies the kind of boundary-sensitive invariant that a stronger negative argument would need.

A Reference pseudocode

The pseudocode below omits symmetry canonicalization and caching, which preserve the answer but are not needed for correctness.

```

MEMBER(r, T):
  T := delete empty components of T
  if T is empty: return true
  if number_of_components(T) > r: return false
  if concatenation(T) is not balanced: return false

  for each choice of one physical endpoint a, one b, one c:
    U := T with these three occurrences deleted
    if MEMBER(r, U): return true

```

```

w := concatenation(T)
for each nonempty proper balanced interval [p,q] of w:
  B := nonempty intersections with the components of T
  A := nonempty outside pieces, preserving boundaries
      and keeping an exposed prefix and suffix separate
  if components(A) <= r and components(B) <= r:
    if MEMBER(r, B) and MEMBER(r, A): return true

if components(T) < r:
  for each split of one component into nonempty x,y:
    U := T with that component replaced by x,y
    if MEMBER(r, U): return true
return false

```

For interval tests, the programs use prefix vectors

$$d_j = (|w[0:j]|_a - |w[0:j]|_c, |w[0:j]|_b - |w[0:j]|_c).$$

The factor $w[p:q]$ is balanced exactly when $d_p = d_q$. Restricting candidate lengths to multiples of three is therefore a safe preliminary filter, not an approximation.

B An explicit positive derivation for the first family member

The following derivation is produced by the family construction. Repeated empty leaves may be shared; the table displays them only once for readability. Each line is an actual tuple in $\mathcal{A}_3$. At the

Step	Tuple	Construction
0	$(\varepsilon, \varepsilon, \varepsilon)$	Empty rule.
1	(a, bc, ε)	Wrap step 0, placing a in the first component.
2	(ε, bc, a)	Wrap step 0, placing a in the last component.
3	$(\varepsilon, b^2c^2, a^2)$	Wrap step 2 at the same three slots.
4	(a, bcb^2c^2, a^2)	Insert step 3 after component two of step 1; regroup.
5	$(a^2, b^2cb^2c^3, a^2)$	Wrap step 4: one a at the first component, b, c at the middle ends.
6	$(a^3, b^3cb^2c^4, a^2)$	Repeat the preceding wrap.

Table 3: A direct derivation of a tuple concatenating to **aaabbbcbcccca**. The JSON certificate uses explicit node IDs.

binary step the combined six-component list is

$$(a, bc, \varepsilon, b^2c^2, a^2, \varepsilon).$$

The zero-based regrouping boundary sequence is $(0, 1, 4, 6)$. The final concatenation is the published length-15 example, while [theorem 5.2](#) supplies its negative level-two proof as a special case.

C Reproduction guide and artifact map

The archive requires Python 3.10 or later, a C++17 compiler for the large enumeration, and a LaTeX installation to rebuild the PDF. The Python tools use only the standard library. Commands below are run from the archive’s top-level directory.

Individual membership queries.

```
python src/mix_parser.py 2 aaabbbcbccccaa
python src/mix_parser.py 3 aaabbbcbccccaa
python src/mix_parser.py 3 a accb b
```

The expected Boolean answers are respectively false, true, and false. Each command-line word after the arity is one component, so the last command is deliberately not a query about a single concatenated word.

Independent checks and exhaustive runs.

```
python tests/forward_oracle.py
python tests/test_results.py
python src/exhaust_python.py 6
mkdir -p build
c++ -O3 -std=c++17 src/exhaust.cpp -o build/mix_exhaust
build/mix_exhaust 7
```

The final argument is the maximum number of occurrences of each letter, not the maximum total word length. Thus 7 means length 21. The C++ driver emits JSON to standard output and progress to standard error. A run interrupted before its final closing JSON braces is not a completed result, even though its header describes the intended complete computation. Increasing the bound can increase time and memory substantially; the programs make no promise of a uniform resource bound in an arbitrary environment.

Positive certificates.

```
python src/check_certificate.py certificates/separation_m1.json
python src/family_certificate.py bbbcbcccc 3 > new_certificate.json
python src/check_certificate.py new_certificate.json
```

The archive includes certificates for $m = 1, 2, 5$ in (6.1). The checker rejects malformed rule parameters or claimed tuples that differ from their computed conclusions.

PDF. Run `make pdf`, or run `pdflatex mix_grammar_report.tex` twice. The bibliography is embedded in the source; a separate BibTeX file is provided for reuse, not required for compilation.

Integrity and verification scope. No checksum manifest is bundled, because re-running the checks or rebuilding the PDF changes file hashes. Hashes would identify the artifacts; they do not prove the code correct. There are no bundled font files, third-party article PDFs, machine-specific executables, or claims of external peer review. The mathematical arguments and programs were produced for this research request; independent human review is appropriate before using any claimed extension as a published result.

References

- [1] Takao Yuyama. *On the Kanazawa–Salvati Conjecture*. arXiv:2609.13871v1 [cs.FL], September 12, 2026, 39 pages. <https://arxiv.org/abs/2609.13871v1>. Relevant locations: Section 1.3 and Example 1.4; Proposition 3.17, Corollary 3.18, and Remark 3.19; Section 6; Proposition E.3 and its proof. Accessed September 20, 2026.
- [2] Makoto Kanazawa and Sylvain Salvati. MIX Is Not a Tree-Adjoining Language. In *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics, Volume 1: Long Papers*, pages 666–674, 2012. <https://aclanthology.org/P12-1070/>.
- [3] Sylvain Salvati. MIX is a 2-MCFL and the word problem in $\mathbb{Z}^2$ is captured by the IO and the OI hierarchies. *Journal of Computer and System Sciences*, 81(7):1252–1277, 2015. DOI: 10.1016/j.jcss.2015.03.004.